

1 **Procedimientos Almacenados**

CREATE PROCEDURE: Define un procedimiento almacenado reutilizable.

```
1 CREATE OR REPLACE PROCEDURE nombre_proc(param1 IN tipo, param2 OUT tipo, param3 IN
2   OUT tipo ) AS
3   -- Declaraciones
4   variable tipo;
5   variable_detabla tabla.tipo%TYPE;
6   CURSOR cursor_name IS SELECT ... FROM ... WHERE ...;
7 BEGIN
8   -- Código del procedimiento
9   open cursor_name;
10  LOOP
11    FETCH cursor_name INTO variable_detabla;
12    EXIT WHEN cursor_name%NOTFOUND;
13    -- Procesar variable_detabla
14  END LOOP;
15  CLOSE cursor_name;
16  SELECT col INTO variable FROM tabla WHERE id = param1;
17  param2 := variable * 2;
18  DBMS_OUTPUT.PUT_LINE('Procedimiento ejecutado');
19 EXCEPTION
20   WHEN NO_DATA_FOUND THEN
21     param2 := 0;
22 /
23 call nombre_proc(1, :var_out, :var_inout);
```

Eliminar procedimiento:

```
1 DROP PROCEDURE nombre_proc;
```

2 **Funciones**

CREATE FUNCTION: Define una función que devuelve un valor.

```
1 CREATE FUNCTION calc_bonus(salario NUMBER, años NUMBER
2 ) RETURN NUMBER AS
3   bonus NUMBER;
4 BEGIN
5   IF años > 5 THEN
6     bonus := salario * 0.15;
7   ELSE
8     bonus := salario * 0.10;
9   END IF;
10  RETURN bonus;
11 EXCEPTION
12   WHEN OTHERS THEN
13     RETURN 0;
14 END;
15 /
```

Uso en consultas:

```
1 SELECT nombre, salario,
2       calc_bonus(salario, años_servicio) AS bonus
3 FROM empleados;
```

Diferencias con procedimientos:

- Función: RETURN obligatorio, puede usarse en SELECT
- Procedimiento: Sin RETURN, solo ejecutable con CALL

3 **Manejo de Excepciones**

Estructura EXCEPTION:

```
1 DECLARE
2   v_salario NUMBER;
3   e_salario_bajo EXCEPTION;
4 BEGIN
5   SELECT salario INTO v_salario FROM empleados WHERE id = 999;
6   IF v_salario < 1000 THEN
7     RAISE e_salario_bajo;
8   END IF;
9 EXCEPTION
10  WHEN NO_DATA_FOUND THEN
11    DBMS_OUTPUT.PUT_LINE('Empleado no encontrado');
12  WHEN TOO_MANY_ROWS THEN
13    DBMS_OUTPUT.PUT_LINE('Múltiples empleados');
14  WHEN OTHERS THEN
15    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
16  WHEN e_salario_bajo THEN
17    DBMS_OUTPUT.PUT_LINE('Salario bajo detectado');
18  WHEN OTHERS THEN
19    DBMS_OUTPUT.PUT_LINE('Otro error: ' || SQLERRM);
20 END;
21 /
```

Excepciones predefinidas:

- NO_DATA_FOUND: SELECT INTO sin resultados.
- TOO_MANY_ROWS: SELECT INTO con múltiples filas.
- ZERO_DIVIDE: División por cero.
- DUP_VAL_ON_INDEX: Violación de clave única.
- INVALID_NUMBER: Conversión numérica inválida.
- VALUE_ERROR: Error aritmético, conversión o truncamiento. OTHERS: Captura cualquier otra excepción.
- RAISE_APPLICATION_ERROR:** Genera error personalizado.

```
1 IF v_edad < 18 THEN
2   RAISE_APPLICATION_ERROR(-20001,
3     'Edad debe ser >= 18');
4 END IF;
```

Rango: -20000 a -20999 para errores de usuario.

4 **Cursorores Avanzados**

Cursor explícito con parámetros:

```
1 DECLARE
2   CURSOR c_emp(dept_id NUMBER, min_sal NUMBER) IS
3     SELECT nombre, salario FROM empleados
4     WHERE departamento_id = dept_id AND salario > min_sal;
5   v_nombre empleados.nombre%TYPE;
6   v_salario empleados.salario%TYPE;
7 BEGIN
8   OPEN c_emp(10, 3000);
9   LOOP
10    FETCH c_emp INTO v_nombre, v_salario;
11    EXIT WHEN c_emp%NOTFOUND;
12    DBMS_OUTPUT.PUT_LINE(v_nombre || ': ' || v_salario);
13  END LOOP;
14  CLOSE c_emp;
15 END;
16 /
```

FOR...LOOP con cursor (más simple):

```
1 BEGIN
2   FOR rec IN (SELECT nombre, salario FROM empleados) LOOP
3     DBMS_OUTPUT.PUT_LINE(rec.nombre || ': ' || rec.salario);
4   END LOOP;
5 END;
6 /
```

Atributos de cursor:

- %FOUND: TRUE si último FETCH exitoso.
- %NOTFOUND: TRUE si último FETCH sin datos.
- %ROWCOUNT: Número de filas procesadas.
- %ISOPEN: TRUE si cursor abierto.

5 **Control Avanzado**

FOR:

```
1 FOR i IN 1..10 LOOP
2   DBMS_OUTPUT.PUT_LINE('Número: ' || i);
3 END LOOP;
```

IF/ELSE IF/ELSE:

```
1 IF condicion1 THEN
2   -- código
3 ELSIF condicion2 THEN
4   -- código
5 ELSE
6   -- código
7 END IF;
```

CASE expresión:

```
1 v_calif := CASE
2   WHEN v_nota >= 90 THEN 'Sobresaliente'
3   WHEN v_nota >= 70 THEN 'Notable'
4   ELSE 'Aprobado' END;
```

Loop con etiquetas:

```
1 <<ext>> FOR i IN 1..10 LOOP
2   <<int>> FOR j IN 1..10 LOOP
3     EXIT ext WHEN cond;
4   END LOOP int;
5 END LOOP ext;
```

CONTINUE:

```
1 FOR i IN 1..100 LOOP
2   CONTINUE WHEN MOD(i,2)=0;
3   procesar(i);
4 END LOOP;
```

6 **Otras funciones notables de PL/SQL**

Para unir textos y devolverlos por pantalla:

```
1 DBMS_OUTPUT.PUT_LINE('Hola, ' || v_nombre);
```

Para modificar datos en tablas:

```
1 UPDATE tabla SET col1 = 'nuevo_valor' WHERE condicion;
2 DELETE FROM tabla WHERE condicion;
3 INSERT INTO tabla (col1, col2) VALUES ('val1', 'val2');
```

7 **Ejemplos Prácticos PL/SQL**

Procedimiento con cursor:

```
1 CREATE PROCEDURE ajustar_salarios(dept_id NUMBER, pct NUMBER) AS
2   CURSOR c_emp IS SELECT emp_id FROM empleados
3   WHERE departamento_id = dept_id FOR UPDATE;
4 BEGIN
5   FOR rec IN c_emp LOOP
6     UPDATE empleados SET salario = salario * (1 + pct/100)
7     WHERE CURRENT OF c_emp;
8   END LOOP;
9   COMMIT;
10 EXCEPTION
11   WHEN OTHERS THEN ROLLBACK;
12 END;
13 /
```

8 **DDL - Definición de Datos**

CREATE TABLE: Crea una nueva tabla.

```
1 create table tabla(
2   col1 tipo primary key,
3   col2 tipo
4 );
```

Tipos: string, int/NUMBER, float, etc.

PRIMARY KEY: Define clave primaria (simple o compuesta).

9 **DML - Manipulación de Datos**

INSERT: Inserta tuplas en una tabla.

```
1 insert into tabla
2   values('val1','val2',123);
3 insert into tabla(col1,col2)
4   values('val1','val2');
```

NULL: Representa valores desconocidos/no aplicables.

10 **Consultas SELECT**

SELECT básico:

```
1 select columnas from tabla
2   where condicion;
```

SELECT *: Selecciona todas las columnas.

DISTINCT: Elimina duplicados en resultados.

11 **Operadores Lógicos**

AND, OR, NOT: Operadores booleanos.

Comparación: =, <>, <, >, <=, >=

IS NULL / IS NOT NULL: Comprueba valores nulos.

IN / NOT IN: Pertenencia a conjunto.

12 VISTAS (CREATE VIEW)

Las vistas son consultas almacenadas que pueden referenciarse como tablas.

```
1 create view nombre_vista as
2 select ... from ... where ...;
```

13 Operaciones de Conjuntos

UNION: Unión de resultados (elimina duplicados). **UNION DISTINCT:** Unión eliminando duplicados explícitamente. **INTERSECT:** Intersección de resultados. **EXCEPT:** Diferencia de conjuntos (A - B). **EXCEPT ALL:** Diferencia manteniendo duplicados.

```
1 select dni from programadores
2 union / union distinct / intersect / except / except all
3 select dni from analistas;
```

14 JOIN - Combinación de Tablas

JOIN implícito (producto cartesiano con WHERE):

```
1 select * from tabla1, tabla2
2 where tabla1.id = tabla2.id;
```

NATURAL JOIN: Join por columnas con mismo nombre.

```
1 select * from tabla1
2 natural join tabla2;
```

JOIN explícito:

```
1 select * from tabla1
2 join tabla2 on tabla1.col1 = tabla2.col2;
```

Se puede usar **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL JOIN**.

15 Funciones de Agregación

SUM(col): Suma de valores.
COUNT(*): Cuenta tuplas.
AVG(col): Media aritmética.
MAX(col): Valor máximo.
MIN(col): Valor mínimo.

```
1 select dni, sum(horas) as total
2 from distribucion
3 group by dni;
```

16 GROUP BY y HAVING

GROUP BY: Agrupa filas por columna(s). **HAVING:** Filtra grupos (después de GROUP BY).

```
1 select col, count(*) from tabla
2 group by col
3 having count(*) > 5;
```

Diferencia: WHERE filtra antes de agrupar, HAVING después.

17 Like

LIKE: Busca patrones en cadenas.

```
1 where nombre like 'A%'; -- Nombres que empiezan por A
2 where nombre like '%son'; -- Nombres que terminan en son
3 where nombre like '%an%'; -- Nombres que contienen 'an'
4 where nombre like '_a__'; -- Nombres de 4 letras con 'a' en 2ª posición
```

18 Subconsultas

Subconsulta escalar: Devuelve un único valor.

```
1 select * from tabla where col =
2 (select max(col) from tabla);
```

Subconsulta en IN:

```
1 where dni in
2 (select dni from analistas);
```

Subconsulta en FROM: Tabla derivada.

```
1 select * from
2 (select dni from prog
3 union select dni from anal);
```

19 DIVISION

Operador específico de DES para encontrar elementos que se relacionan con todos los de otro conjunto.

```
1 select dniEmp from
2 (select codigoPr,dniEmp
3 from distribucion)
4 division
5 (select codigoPr from dist
6 where dniEmp='5');
```

Interpretación: Empleados asignados a todos los proyectos del empleado '5'.

20 Operaciones Aritméticas

Se pueden realizar operaciones en SELECT:

```
1 select codigo, horas*1.2
2 from distribucion;
```

Operadores: +, -, *, /

21 Alias y RENAME

AS: Define alias para columnas/tablas.

```
1 select dni as identificador,
2 sum(horas) as total
3 from distribucion;
```

RENAME (sintaxis DES):

```
1 rename (dniEmp as dni)
2 (distribucion);
```

22 WITH - Expresiones de Tabla Comunes (CTE)

WITH ... AS permite definir tablas temporales nombradas (CTE) que simplifican consultas complejas y mejoran legibilidad. Se pueden encadenar varias CTE y usar **WITH RECURSIVE** para recursión.

Sintaxis básica:

```
1 WITH nombre_cte (columna1, ...) AS (
2 select ...
3 )
4 SELECT ... FROM nombre_cte;
```

Ejemplo 1: sumar horas por empleado y filtrar

```
1 WITH total_horas (dniEmp, total) AS (
2 SELECT dniEmp, SUM(horas) AS total
3 FROM distribución
4 GROUP BY dniEmp
5 )
6 SELECT dniEmp, total
7 FROM total_horas
8 WHERE total > 20;
```

Ejemplo 2: equivalente a DIVISION (empleados que hacen todos los proyectos de 'Evaristo')

```
1 WITH proyectos_eva (codigoPr) AS (
2 SELECT codigoPr
3 FROM distribución
4 WHERE dniEmp = (SELECT dni FROM analistas WHERE nombre = 'Evaristo')
5 ),
6 empleados_conjunto (dniEmp) AS (
7 SELECT dniEmp
8 FROM distribución
9 WHERE codigoPr IN (SELECT codigoPr FROM proyectos_eva)
10 GROUP BY dniEmp
11 HAVING COUNT(DISTINCT codigoPr) = (SELECT COUNT(*) FROM proyectos_eva)
12 )
13 SELECT dniEmp FROM empleados_conjunto;
```

23 Uso del TOP y ORDER BY

TOP n: Limita el número de filas devueltas. **ORDER BY:** Ordena resultados por columna(s), se puede usar **ASC** (ascendente) o **DESC** (descendente).

```
1 SELECT TOP 5 dniEmp, SUM(horas) AS total
2 FROM distribución
3 GROUP BY dniEmp
4 ORDER BY total DESC;
```

24 Ejemplos Prácticos

Empleados sin teléfono:

```
1 select dni,nombre from prog
2 where telefono is null
3 union
4 select dni,nombre from anal
5 where telefono is null;
```

Suma de horas por empleado:

```
1 select dni, sum(horas) as total
2 from (select dni from prog
3 union select dni from anal)
4 join dist on dni = dniEmp
5 group by dni;
```

Proyectos sin analistas:

```
1 select codigo from proyectos
2 except
3 select distinct codigo from
4 (select codigoPr as codigo,
5 dniEmp from dist)
6 join analistas on dniEmp = dni;
```

Combinando múltiples operaciones:

```
1 create view vista_compleja as
2 (select dni,nombre,codigoPr
3 from (select * from prog
4 union select * from anal)
5 join dist on dni = dniEmp)
6 union
7 (select dni,nombre,codigo
8 from (select * from prog
9 union select * from anal)
10 join proy on dni = dniDir);
```

Combinando JOIN y WITH:

```
1 WITH pelea (barco, fecha, resultado) AS (
2 SELECT barco, fecha, resultado
3 FROM batallas join resultados
4 ON nombre = batalla
5 )
6 SELECT barco
7 FROM pelea p1 JOIN pelea p2
8 ON p1.barco = p2.barco
9 WHERE p1.fecha < p2.fecha
10 AND p1.resultado = 'victoria'
11 AND p2.resultado = 'derrota';
```

Uso de 2 JOIN simultaneos

```
1 SELECT PROJNO, DEPTNAME, MGRNO, LASTNAME
2 FROM PROJECT JOIN DEPARTMENT
3 ON PROJECT.DEPTNO = DEPARTMENT.DEPTNO
4 JOIN EMPLOYEE
5 ON DEPARTMENT.MGRNO = EMPLOYEE.EMPNO
6 WHERE DEPARTMENT.DEPTNO = 'D21'
7 ORDER BY PROJNO;
```

Que equivale a:

```
1 SELECT PROJNO, DEPTNAME, MGRNO, LASTNAME
2 FROM PROJECT P JOIN DEPARTMENT D
3 ON P.DEPTNO = D.DEPTNO
4 JOIN EMPLOYEE E
5 ON D.MGRNO = E.EMPNO
6 WHERE D.DEPTNO = 'D21'
7 ORDER BY P.PROJNO;
```

En este ultimo caso, se puede poner en el **SELECT PROJNO** o **P.PROJNO**, ya que no hay ambigüedad.